

Sistemas Distribuidos

GRADO EN INGENIERÍA INFORMÁTICA



MEMORIA

Ejercicio evaluable 2: Sistemas Distribuidos

Curso 2025/2026

DATOS DE LOS AUTORES

Nombre:	David Sierra
NIA:	100471913
Correo electrónico:	100471913@alumnos.uc3m.es

Nombre:	Juan Luis del Valle Caetano
NIA:	100454201
Correo electrónico:	100454201@alumbos.uc3m.es

1. Visión general

El objetivo es implementar el servicio de tuplas <key, value1, value2, value3> del EJ1, sustituyendo la comunicación local por sockets TCP. Los ficheros claves.h, claves.c y app-cliente.c se reutilizan íntegramente. Solo se implementan proxy-sock.c y servidor-sock.c.

La arquitectura se compone de:

Archivo	Función
proxy-sock.c → libproxyclaves.so	Implementa la API del cliente; abre conexión TCP por operación y serializa los mensajes.
servidor-sock.c → servidor_sock	Servidor concurrente (hilo por conexión); deserializa mensajes y llama a claves.c.
claves.c → libclaves.so	Lógica de almacenamiento con lista enlazada dinámica y mutex.
app-cliente.c → app-cliente-sock	Batería de pruebas; solo usa la API pública de claves.h.

2. Estructura de almacenamiento

Se utiliza una lista enlazada dinámica en memoria compartida entre hilos, protegida con un mutex de POSIX. No impone límite de elementos y no requiere software adicional. Cada nodo contiene:

Campo	Tipo / Restricción
key	char[] — hasta 255 caracteres, único en la lista.
value1	char[] — hasta 255 caracteres.
value2	float[] — N floats ($1 \leq N \leq 32$).
value3	struct Paquete { int x, y, z; }.

3. Concurrencia

El servidor crea un hilo POSIX por cada conexión aceptada (pthread_create en modo detached). El hilo atiende completamente al cliente y termina de forma autónoma. El acceso a la lista enlazada se serializa con un mutex global en claves.c, garantizando la consistencia ante múltiples clientes concurrentes.

4. Protocolo de aplicación (proxy ↔ servidor)

4.1 Principios de diseño

El protocolo es textual y basado en líneas (). No se transmiten estructuras C en crudo ni se asume ningún layout de memoria, lo que garantiza la independencia del lenguaje de programación.

Flujo por operación: el proxy abre una nueva conexión TCP → envía la petición → lee la respuesta → cierra la conexión.

4.2 Formato de mensajes

Operación	Petición (proxy → servidor)	Respuesta (servidor → proxy)
DESTROY	DESTROY\n	0\n ó -1\n
SET_VALUE	SET_VALUE\n key\n value1\n N\n f1 f2 ... fN\n x\n y\n z\n	0\n ó -1\n
GET_VALUE	GET_VALUE\n key\n	resultado\n [value1\n N\n f1...fN\n x\n y\n z\n] (si resultado = 0)

Operación	Petición (proxy → servidor)	Respuesta (servidor → proxy)
MODIFY_VALUE	MODIFY_VALUE\n key\n value1\n N\n f1 f2 ... fN\n x\n y\n z\n	0\n ó -1\n
DELETE_KEY	DELETE_KEY\n key\n	0\n ó -1\n
EXIST	EXIST\n key\n	1\n (existe) ó 0\n (no existe) ó -1\n

4.3 Semántica de códigos de retorno

Código	Significado
0	Operación completada con éxito.
-1	Error lógico: clave inexistente o duplicada, N fuera de rango, etc.
-2	Error de comunicación: fallo de socket, conexión rechazada, respuesta mal formada.

5. Compilación y ejecución

5.1 Variables de entorno (lado cliente)

Variable	Descripción
IP_TUPLAS	Dirección IP o nombre del servidor (decimal-punto o dominio-punto).
PORT_TUPLAS	Puerto TCP del servidor.

5.2 Compilación

Compilar todo el proyecto:

```
make
```

Artefactos generados: libclaves.so, libproxyclaves.so, app-cliente, app-cliente-sock, servidor_sock.

5.3 Ejecución

Servidor: `./servidor_sock <PUERTO>` Ej: `./servidor_sock 4500`

Cliente: `export IP_TUPLAS=localhost PORT_TUPLAS=4500 && ./app-cliente-sock`

(Alternativa) `env IP_TUPLAS=localhost PORT_TUPLAS=4500 ./app-cliente-sock`

6. Plan de pruebas

El cliente app-cliente.c ejecuta 15 pruebas que cubren funcionalidad correcta, casos de error, límites de entrada y consistencia tras operaciones encadenadas:

#	Prueba	Descripción	Esperado
1	destroy() inicial	Limpia el estado del servidor.	0
2	set_value (x2 válidas)	Inserta dos tuplas con datos correctos.	0
3	set_value clave duplicada	Intento de inserción con clave ya existente.	-1
4	set_value N fuera de rango	N=0 y N=33 deben ser rechazados.	-1
5	exist clave existente/inexistente	Comprueba presencia de clave.	1 / 0
6	get_value + verificación	Recupera y verifica value1, vector y estructura.	0
7	get_value clave inexistente	Clave no presente en el sistema.	-1
8	modify_value + verificación	Modifica y verifica los nuevos valores.	0
9	modify_value clave inexistente	Clave no presente en el sistema.	-1
10	delete_key + exist tras borrar	Borra y comprueba que ya no existe.	0 / 0
11	delete_key clave inexistente	Clave no presente en el sistema.	-1
12	Límites de cadenas	Clave 256 chars → error; clave 255 chars → válida.	-1 / 0

#	Prueba	Descripción	Esperado
13	set_value y get_value N=32	Vector de tamaño máximo permitido.	0
14	10 inserciones + exist	Inserción múltiple y verificación de existencia.	0
15	destroy() final	Limpieza total y verificación de estado vacío.	0

7. Conclusión

La solución mantiene la misma API del ejercicio 1, separando limpiamente la lógica de almacenamiento (claves.c) del transporte (proxy-sock.c y servidor-sock.c). El protocolo textual basado en líneas garantiza la independencia del lenguaje de programación. La configuración mediante variables de entorno evita direcciones embebidas en el código y facilita el despliegue en distintos entornos sin recompilar. El servidor concurrente basado en hilos POSIX bajo demanda permite atender varios clientes simultáneamente, manteniendo la seguridad del acceso a datos gracias al mutex de claves.c